DEEPGROUNDREPAIR: A Tool for API-Grounded Generation and Shape-Aware Repair of Deep Learning Programs

Xiangyue Ma, Zhen Wang, Jun Li, Haotian Li, and Zheng Zheng
Beihang University, Beijing, China

***Abstract*—Large language models (LLMs) can generate deep learning (DL) programs from natural-language requirements, but the generated programs may contain API-level syntactic issues or fail at runtime because of tensor-shape mismatches. We present the tool realization of DEEPGROUNDREPAIR, which integrates API-grounded generation, failure localization, and shape-aware repair into an executable workflow. It grounds draft generation in task-relevant API knowledge, localizes upstream API configurations behind tensor-shape failures, and uses runtime tensor shapes to guide parameter repair. Re-execution validates the resulting program. DEEPGROUNDREPAIR supports TensorFlow, PyTorch, and PaddlePaddle and is intended for DL developers and software engineering researchers working with LLM-generated DL code. The artifact is available at https://github.com/maxy-635/DeepGroundRepair; the demonstration video link will be added in the camera-ready version.**

***Index Terms*—deep learning programs, large language models, code generation, failure localization, program repair**

I. INTRODUCTION

Deep learning (DL) programs must satisfy library-specific API usage constraints and tensor-shape dependencies. LLM-generated DL programs may therefore contain nonexistent or misused APIs and may also fail at runtime because of tensor-shape mismatches. Such failures are difficult to diagnose because the operation reporting a mismatch can be downstream from the API configuration that introduced the incompatible shape.

Existing retrieval-augmented generation (RAG) and execution-feedback-guided repair methods face DL-specific challenges. RAG methods struggle to retrieve relevant APIs due to the gap between high-level task requirements and fine-grained API knowledge [1]. Meanwhile, traceback-guided repair provides insufficient guidance for resolving tensor-shape mismatches, which account for 77.34% of runtime failures on average in our prior study [2], as their root causes often lie in upstream API configurations and involve complex tensor-shape dependencies. These limitations motivate a DL-specific tool that grounds code generation in verified API knowledge and guides repair through root-cause localization and explicit tensor-shape constraints.

To address these challenges, we present DEEPGROUNDREPAIR, a tool that integrates API-grounded generation, root-cause localization, and shape-aware repair into a unified *Generate* $\rightarrow$ *Localize* $\rightarrow$ *Repair* workflow. Given a natural-language requirement, a target DL library, and an LLM configuration, the tool generates a draft program grounded in retrieved API knowledge. If execution fails, it localizes the root cause using runtime tensor-shape information and performs targeted repair followed by execution validation. The tool also exposes intermediate diagnostic and repair artifacts, allowing users to inspect the generation and repair process.

This paper presents the tool realization of the previously proposed DEEPGROUNDREPAIR approach. Rather than restating the underlying algorithms, we focus on how its generation–localization–repair backend is used in practice, demonstrated on a representative tensor-shape failure.

II. DEEPGROUNDREPAIR TOOL DESIGN

DEEPGROUNDREPAIR integrates API-grounded generation, failure diagnosis, and shape-aware repair into a unified workflow. As shown in Figure 1, the tool organizes these stages into an executable pipeline, where generated code and diagnostic information are passed between stages to guide subsequent processing. The workflow is supported by locally deployed LLMs and a library-specific API knowledge base covering TensorFlow, PyTorch, and PaddlePaddle.

API-Grounded Generation. Given a natural-language task requirement and a target DL library, DEEPGROUNDREPAIR recommends task-relevant APIs, verifies their existence, and retrieves corresponding API documentation from the knowledge base. The retrieved knowledge is then provided to the LLM to generate a draft DL program. This stage produces the initial code for subsequent execution and diagnosis.

Failure Diagnosis. The tool executes the generated program and captures runtime failures. For tensor-shape mismatches, it combines traceback information with static and dynamic analysis to trace the failure to its upstream API configuration. The diagnosis identifies the root-cause API, suspicious parameters, and relevant tensor shapes. It then masks the suspicious parameters in the generated code, producing parameter-masked code for subsequent repair.

Shape-Aware Repair. The tool annotates the parameter-masked code with traced tensor shapes and retrieves relevant shape-related API knowledge. The annotated code and retrieved knowledge are provided to the LLM to guide targeted repair of the identified parameters. The resulting program is re-executed to validate whether the runtime failure has been resolved.

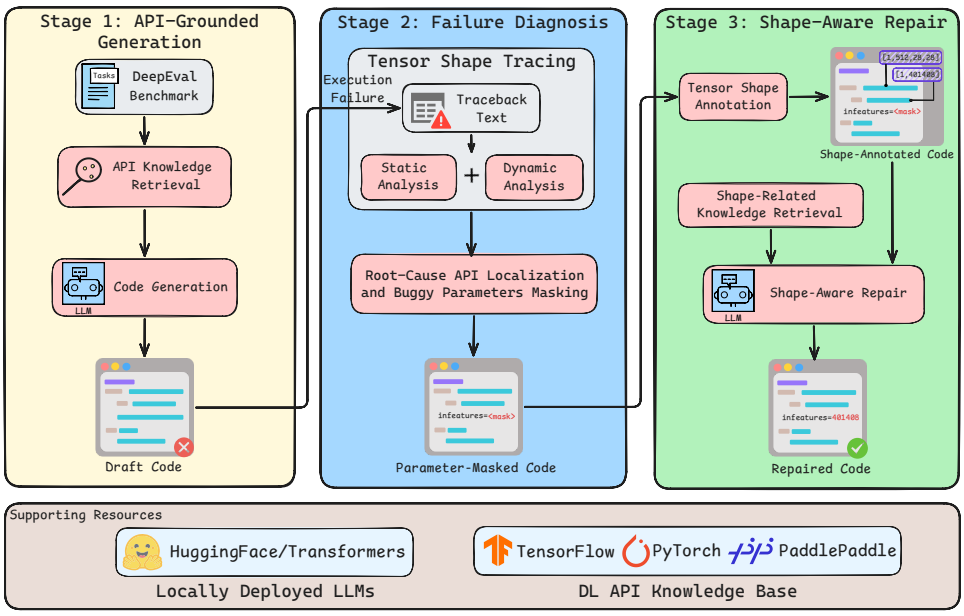


Fig. 1. DEEPGROUNDREPAIR tool workflow.

III. TOOL USAGE

A. Supporting Resources

Implementation and Installation. DEEPGROUNDREPAIR is implemented in Python and supports TensorFlow, PyTorch, and PaddlePaddle. Our experiments were conducted on a server equipped with two NVIDIA V100 GPUs, running CentOS Linux 7.8, Python 3.10, CUDA 12.1, and cuDNN 8.9.5. The DL library versions were TensorFlow 2.16.1, PyTorch 2.11.0, and PaddlePaddle 3.3.0. The artifact provides a Conda environment specification (`environment.yaml`) for installing the required dependencies, with detailed installation instructions available in the artifact repository.

LLM Deployment. DEEPGROUNDREPAIR supports locally deployed open-weight LLMs through Hugging Face Transformers. The current implementation uses models from the Gemma-3 and Mistral families, covering parameter sizes from 3B to 27B. Users can configure the local model checkpoints in the provided execution scripts to perform API recommendation, code generation, and shape-aware repair.

DL API Knowledge Base. DEEPGROUNDREPAIR incorporates library-specific API knowledge bases constructed from the official documentation of TensorFlow, PyTorch, and PaddlePaddle, containing 3,686, 4,137, and 1,531 API entries, respectively. Each knowledge base provides sparse and dense retrieval indexes to support Stages 1 and 3. Specifically, Stage 1 retrieves API names, signatures, and descriptions for draft code generation, while Stage 3 additionally retrieves parameter descriptions and available tensor-shape transformation formulas to guide inter-path mismatch repair.

B. Tool Configuration and Execution

Input and Configuration. The current prototype uses natural-language task requirements from the DeepEval benchmark [2], stored as YAML files under `benchmark/DeepEval/`, each containing a `Requirement` field. Before execution, users configure the target DL library (`--dlls`), benchmark (`--benchmark`), LLM checkpoint, experiment ID, and number of generation repeats (`--repeats`) in the provided scripts. Consistent model and experiment IDs are maintained across the three stages to ensure correct access to intermediate results. The repository also provides preconstructed API knowledge bases under `database/`.

Tool Execution. The three stages are invoked sequentially through their respective `run_main.sh` scripts under `run_bashes/step_one/`, `run_bashes/step_two/`, and `run_bashes/step_three/`. Stage 1 generates draft DL programs and saves them as `generated_code.py`, together with API retrieval records. Stage 2 processes the generated programs to diagnose execution failures, producing `annotated_code.py` and `annotated_code.json`, which contain code and structured diagnostic information for downstream repair. Stage 3 consumes these two artifacts to generate `repaired_code.py` and re-executes the repaired program for validation. The stage-specific code artifacts are organized under `response/`, with execution logs stored under `logs/`. The repository also provides separate evaluation scripts for checking code executability, API-level syntactic issues, and semantic consistency.

IV. EVALUATION AND REPRESENTATIVE DEMONSTRATION

A. Evaluation

The associated research study evaluates DEEPGROUNDREPAIR on 100 model-level DL programming tasks using six open-weight LLMs across TensorFlow, PyTorch, and PaddlePaddle [3]. We compare it with four

TABLE I
API-LEVEL SYNTACTIC ISSUES PER 10 PROGRAMS, AVERAGED ACROSS SIX LLMS (LOWER IS BETTER).

Method	TensorFlow	PyTorch	PaddlePaddle
B1	5.82	3.14	27.35
B2	4.44	1.36	11.09
B3	5.38	1.86	22.42
B4	4.28	**1.08**	9.88
DeepGR	**0.94**	1.30	**6.08**

TABLE II
CODE EXECUTABILITY RATE (CER, %) AVERAGED ACROSS SIX LLMS (HIGHER IS BETTER).

Method	TensorFlow	PyTorch	PaddlePaddle
B1	37.8	17.0	12.2
B2	36.7	22.7	8.3
B3	51.2	26.5	19.5
B4	49.0	31.3	17.8
DeepGR	**58.2**	**41.2**	**31.5**

baselines: Vanilla LLM (B1), Hybrid-RAG (B2), and their traceback-guided self-debugging counterparts (B3 and B4).

End-to-End Effectiveness. We use code executability, measured by Code Executability Rate (CER), as the primary effectiveness criterion because successful execution is a fundamental requirement for generated DL programs. As shown in Table I, DEEPGROUNDREPAIR reduces API-level syntactic issues, including API hallucinations and parameter misuses, by an average of 5.4 issues per 10 programs compared with the four baselines. As shown in Table II, it achieves the highest average CER across all three DL libraries, with an average relative improvement of 86.0% over the baselines. Compared with the two self-debugging baselines, it further improves Semantic Consistency Rate (SCR) by an average of 16.7%, indicating better preservation of program structure during repair.

Token Efficiency. Compared with the generation-only baselines (B1–B2), DEEPGROUNDREPAIR incurs 25.0%–63.0% higher token cost while improving CER by 54.0%–279.5%. More notably, compared with the self-debugging baselines (B3–B4), it reduces token cost by 18.5%–33.9% while improving CER by 13.7%–77.0%. These results demonstrate that DEEPGROUNDREPAIR achieves higher executability with lower token cost than traceback-guided self-debugging approaches. Further experimental results and detailed analyses are presented in the associated research study [3].

B. Representative Demonstration

Figure 2 illustrates the generation–localization–repair workflow using a representative PyTorch tensor-shape mismatch from the associated study [3].

(a) Draft Code with Bug. As shown in Figure 2(a), the draft program defines a fully connected layer using `nn.Linear` with `in_features=512*7*7` and `out_features=1024`. However, the actual input tensor to this layer contains 401,408 features, rather than the configured 25,088. Consequently, execution fails at `self.fc1(x)` with a matrix-shape mismatch.

(b) Parameter-Masked Code. As shown in Figure 2(b), DEEPGROUNDREPAIR combines the traceback with static and dynamic analysis to trace the failure from `self.fc1(x)` back to the upstream `nn.Linear` constructor. It identifies `in_features` as the suspicious parameter and replaces its value with `<mask>`, producing parameter-masked code for subsequent repair.

(c) Shape-Annotated Code. As shown in Figure 2(c), the tool annotates the parameter-masked code with runtime tensor shapes. Specifically, the tensor shape changes from `(1,512,28,28)` to `(1,401408)` after the flattening operation. These annotations provide explicit shape evidence for inferring the correct input dimension of the fully connected layer.

(d) Repaired Code. As shown in Figure 2(d), DEEPGROUNDREPAIR uses the annotated tensor shapes to infer the correct `in_features` value and replaces the masked parameter with `401408`. The repaired program is then re-executed to validate whether the tensor-shape mismatch has been resolved.

V. RELATED WORK

Bugs and Constraints in Deep Learning Programs. Deep learning programs compose library-specific APIs into computation graphs, so correctness depends on both valid API usage and compatible tensor transformations. Empirical evidence on LLM-generated DL programs reports frequent API hallucinations, parameter misuses, and runtime tensor-shape mismatches; the latter constitute the predominant class of execution failures in the studied settings [2]. These observations motivate tools that expose DL-specific API and shape constraints during both generation and repair.

Retrieval-Augmented Code Generation. Retrieval-augmented generation (RAG) grounds an LLM's output in external knowledge [1]. API documentation can provide signatures, parameter descriptions, and usage constraints, while relevant code retrieval can provide reusable implementation context. However, a natural-language DL requirement describes modeling intent at a higher level than an individual API, so a direct retrieval query may return irrelevant documentation. DEEPGROUNDREPAIR addresses this gap by recommending and verifying task-relevant APIs before retrieving library-specific knowledge for generation and repair.

Repair of LLM-Generated Code. Feedback-guided methods use execution results or self-generated explanations to iteratively revise generated programs; self-debugging is a representative example [4]. Such feedback is useful for general code repair, but a raw traceback often identifies only the downstream operation where a DL shape mismatch becomes visible. DEEPGROUNDREPAIR complements feedback-guided repair with root-cause localization and explicit runtime tensor-shape guidance, thereby targeting the upstream API configuration

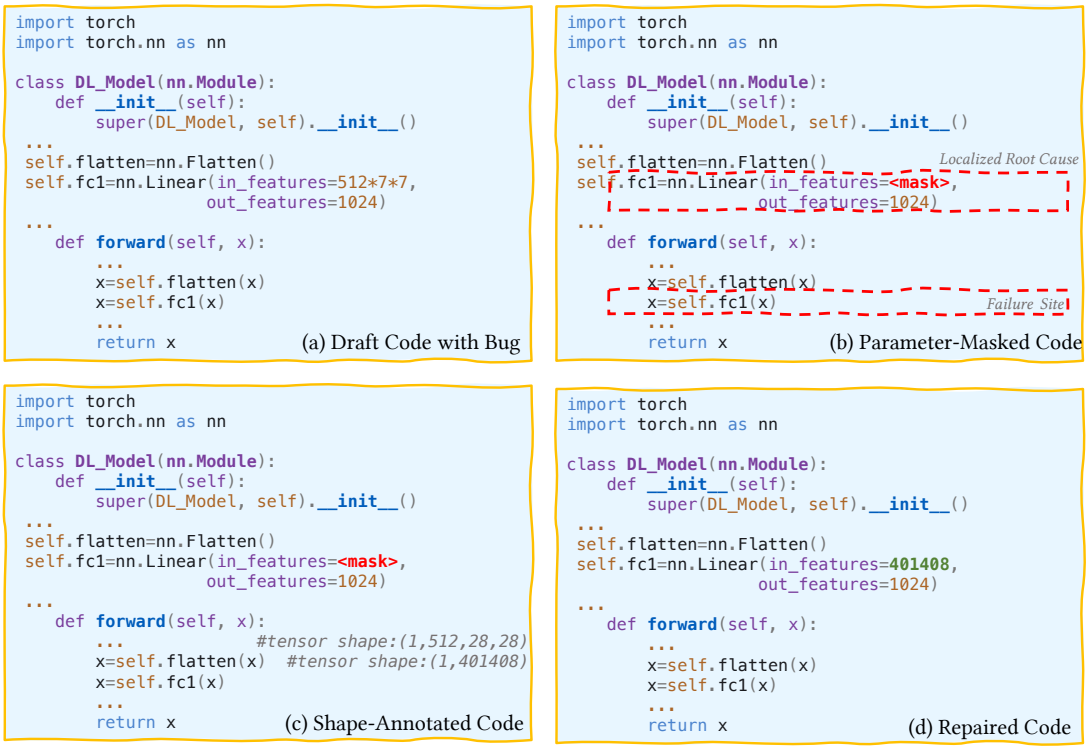


Fig. 2. Example of .

and parameter values that need to be changed. The present paper focuses on the executable realization and usage of this generation–localization–repair workflow, while the underlying framework is described in the associated research study [3].

VI. Conclusion

This tool paper presents the executable realization of DeepGroundRepair for LLM-generated deep learning programs. Starting from a natural-language requirement, the tool grounds draft generation in library-specific API knowledge, diagnoses the upstream causes of tensor-shape failures, produces a shape-aware repair, and validates the repaired program through re-execution. The current implementation supports TensorFlow, PyTorch, and PaddlePaddle, and exposes intermediate artifacts such as retrieved APIs, failure traces, localized parameters, runtime shapes, and repaired code so that users can inspect and reproduce the workflow.

Across the evaluated model-level tasks, the associated study reports fewer API hallucinations and parameter misuses, an average relative improvement of 86.0% in code executability, and a 16.7% average improvement in semantic consistency over traceback-guided self-debugging baselines [3]. These results support the practical value of combining API grounding, root-cause localization, and shape-aware repair in one tool workflow. Future work will extend the tool with lightweight constraint checking during generation, stronger analysis for complex tensor flows, adaptive multi-step repair for interacting faults, and diagnosis of additional failure types such as dtype or device incompatibilities, numerical instability, and training-time failures.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020.

[2] X. Ma, X. Du, C. Li, J. Meng, X. Fang, W. Ding, and Z. Zheng, "How do large language models perform in deep learning code generation? an empirical study," *ACM Trans. Softw. Eng. Methodol.*, May 2026, just Accepted. [Online]. Available: https://doi.org/10.1145/3816024

[3] X. Ma, Z. Wang, X. Lu, X. Du, C. Li, J. Li, H. Li, and Z. Zheng, "DeepGroundRepair: From API-grounded generation to shape-aware repair for deep learning programs," 2026. [Online]. Available: https://arxiv.org/abs/2610.16897

[4] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations*, 2024.